

Jour 1 : Architecture des Pipelines CI/CD - Principes et GitLab CI Avance

Table des matieres

1. [Introduction au CI/CD](#)
2. [Le workflow CI/CD explique simplement](#)
3. [Les concepts cles](#)
4. [Le fichier .gitlab-ci.yml](#)
5. [Le principe Fail Fast](#)
6. [La parallelisation](#)
7. [Artifacts et Cache](#)
8. [Extends et Anchors YAML](#)
9. [Include : modularisation](#)
10. [La pyramide de tests](#)
11. [Les patterns de deploiement](#)
12. [Les rules conditionnelles](#)
13. [Recapitulatif et checklist](#)

1. Introduction au CI/CD

Qu'est-ce que le CI/CD ?

Imaginez que vous travaillez en equipe sur un projet. Chaque developpeur ecrit du code de son cote. A un moment, il faut rassembler tout ce code, verifier qu'il fonctionne, et le mettre en ligne pour les utilisateurs.

Sans CI/CD :

- Un developpeur finit son code le vendredi soir
- Il envoie son code par email ou cle USB au responsable
- Le responsable essaie de fusionner tout le code le week-end
- Lundi matin : rien ne fonctionne, c'est le chaos
- Le deploiement prend 3 jours de corrections manuelles

Avec CI/CD :

- Un developpeur pousse son code sur GitLab

- Automatiquement, le code est verifie, teste et deploye
- En quelques minutes, on sait si tout fonctionne
- Si un probleme est detecte, le developpeur est prevenu immediatement

Les deux parties du CI/CD

CI = Continuous Integration (Integration Continue)

C'est le processus automatique qui :

1. Recupere le nouveau code pousse par un developpeur
2. Le compile (si necessaire)
3. Execute les tests automatiques
4. Verifie la qualite du code (linting)
5. Signale immediatement les problemes

Analogie : C'est comme un correcteur automatique qui relit votre texte des que vous l'ecrivez, et souligne les erreurs en rouge immediatement.

CD = Continuous Delivery / Deployment (Livraison/Deploiement Continu)

C'est le processus automatique qui :

1. Prend le code valide par la CI
2. Le prepare pour la mise en production
3. Le deploie sur les serveurs (staging, production)
4. Verifie que le deploiement s'est bien passe

Analogie : C'est comme un service de livraison qui, des que votre colis est emballe et verifie, le livre automatiquement a l'adresse du destinataire.

Pourquoi le CI/CD est indispensable ?

Sans CI/CD	Avec CI/CD
Deploiements risques et stressants	Deploiements frequents et sereins
Bugs decouverts tard (en production)	Bugs decouverts tot (avant la production)
Integration du code = cauchemar	Integration du code = automatique
"Ca marchait sur ma machine"	Ca marche partout pareil
Heures de travail manuel	Tout est automatise

Les outils CI/CD populaires

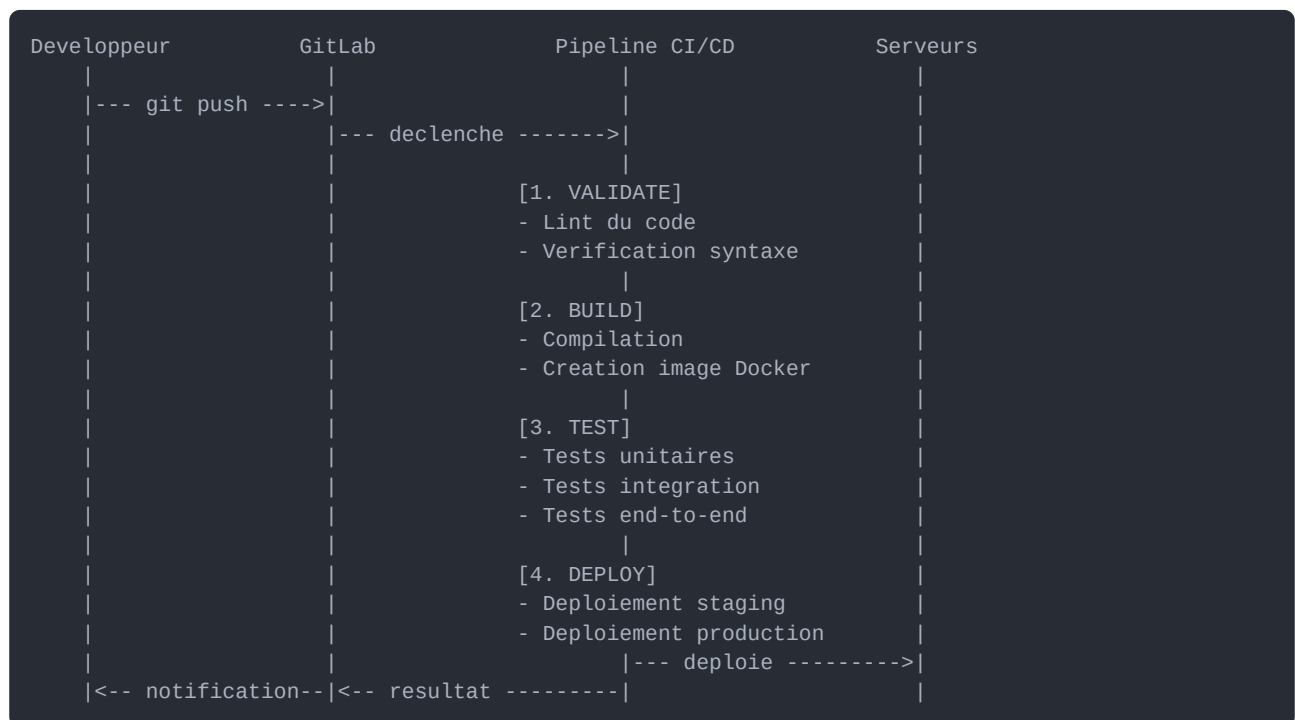
Outil	Particularite
-------	---------------

GitLab CI	Integre directement dans GitLab (notre choix pour ce cours)
GitHub Actions	Integre dans GitHub
Jenkins	Le plus ancien, tres flexible
CircleCI	Cloud, simple a configurer
Travis CI	Populaire pour l'open source
Azure DevOps	Ecosysteme Microsoft

Nous utilisons **GitLab CI** car il est integre directement dans GitLab :
pas besoin d'installer un outil externe, tout est au meme endroit.

2. Le workflow CI/CD explique simplement

Le schema global



Explication etape par etape

Etape 1 - Le developpeur pousse son code :

```

# Le developpeur a fini une fonctionnalite
git add .           # Ajoute les fichiers modifies
git commit -m "Ajout login" # Cree un commit avec un message
git push origin ma-branche # Envoie le code sur GitLab
  
```

Etape 2 - GitLab detecte le push :

GitLab voit qu'un nouveau code est arrive. Il cherche un fichier `.gitlab-ci.yml` a la racine du projet. Ce fichier contient les instructions du pipeline.

Etape 3 - Le pipeline s'exécute :

Chaque etape (stage) s'exécute dans l'ordre. Si une etape echoue, les suivantes ne s'exécutent pas (par défaut).

Etape 4 - Notification :

Le developpeur recoit une notification : succes ou echec, avec les details.

3. Les concepts cles

3.1 Pipeline

Un **pipeline** est l'ensemble du processus CI/CD qui s'exécute quand vous poussez du code.

```
Pipeline = Stage 1 --> Stage 2 --> Stage 3 --> Stage 4
            (validate)  (build)   (test)   (deploy)
```

- Un pipeline est declenche par un evenement (push, merge request, schedule...)
- Un pipeline contient un ou plusieurs **stages** (etapes)
- Si un stage echoue, le pipeline s'arrete (sauf configuration speciale)

3.2 Stage (Etape)

Un **stage** est une etape dans le pipeline. Les stages s'exécutent dans l'ordre.

```
# Definition de l'ordre des stages
stages:
- validate    # S'exécute en premier
- build       # S'exécute en deuxieme (si validate reussit)
- test        # S'exécute en troisieme (si build reussit)
- deploy      # S'exécute en dernier (si test reussit)
```

Important : Tous les jobs d'un meme stage s'exécutent EN PARALLELE.
Les stages entre eux s'exécutent EN SEQUENCE.

3.3 Job

Un **job** est une tache specifique a l'interieur d'un stage.

```
# Ceci est un job
lint-code:      # Nom du job (vous choisissez)
  stage: validate # A quel stage il appartient
  script:        # Les commandes a executer
    - npm run lint      # Commande 1
    - npm run format    # Commande 2
```

Regles importantes pour les jobs :

- Chaque job a un **nom unique**
- Chaque job appartient a un **stage**
- Chaque job a un **script** (les commandes a executer)
- Les jobs d'un meme stage s'executent en parallele
- Si un job echoue, tout le stage echoue

3.4 Runner

Un **runner** est la machine qui execute les jobs.

```
Pipeline --> Runner --> Execute les commandes du job
```

Types de runners :

- **Shared runners** : fournis par GitLab.com, partages entre tous les projets
- **Group runners** : dedies a un groupe de projets
- **Specific runners** : dedies a un seul projet

Analogie : Le pipeline est la recette de cuisine, le job est une etape de la recette, et le runner est le cuisinier qui fait le travail.

3.5 Variables CI/CD

Les **variables** permettent de stocker des valeurs reutilisables.

```
# Variables definies dans le fichier .gitlab-ci.yml
variables:
  NODE_VERSION: "18"      # Version de Node.js
  APP_NAME: "mon-application" # Nom de l'application
  DEPLOY_PATH: "/var/www/app" # Chemin de deploiement

# Utilisation dans un job
build:
  script:
    - echo "Construction de $APP_NAME" # Utilise la variable
    - echo "Avec Node.js $NODE_VERSION" # Utilise la variable
```

Il existe aussi des **variables predefinies** par GitLab :

- **\$CI_COMMIT_SHA** : le hash du commit
- **\$CI_COMMIT_BRANCH** : le nom de la branche

- `$CI_PIPELINE_ID` : l'identifiant du pipeline
- `$CI_PROJECT_NAME` : le nom du projet
- `$CI_JOB_NAME` : le nom du job en cours

4. Le fichier `.gitlab-ci.yml`

4.1 Qu'est-ce que YAML ?

YAML (Yet Another Markup Language) est un format de fichier pour écrire des configurations. C'est comme du JSON mais plus lisible par les humains.

Règles de base du YAML :

```
# Ceci est un commentaire (commence par #)

# Une cle et une valeur simple
nom: "Jean"

# Un nombre (pas besoin de guillemets)
age: 25

# Un booléen
actif: true

# Une liste (chaque élément commence par un tiret)
fruits:
  - pomme
  - banane
  - orange

# Un objet imbriqué (indentation de 2 espaces)
adresse:
  rue: "12 rue de Paris"
  ville: "Lyon"
  code_postal: 69000

# ATTENTION : l'indentation est CRUCIALE en YAML
# Utilisez TOUJOURS des espaces, JAMAIS des tabulations
# L'indentation standard est de 2 espaces
```

4.2 Structure de base d'un `.gitlab-ci.yml`

```
# =====
# FICHER : .gitlab-ci.yml
# Ce fichier DOIT etre a la RACINE de votre projet
# C'est lui qui dit a GitLab quoi faire quand vous poussez du code
# =====

# --- IMAGE PAR DEFAULT ---
# L'image Docker utilisee pour executer les jobs
# C'est comme choisir quel "ordinateur" va faire le travail
image: node:18-alpine

# --- DEFINITION DES STAGES ---
# L'ordre dans lequel les etapes vont s'executer
# C'est comme un plan de route : etape 1, puis 2, puis 3...
stages:
  - validate    # Etape 1 : verifier la qualite du code
  - build       # Etape 2 : construire l'application
  - test        # Etape 3 : tester l'application
  - deploy      # Etape 4 : deployer l'application

# --- VARIABLES GLOBALES ---
# Des valeurs reutilisables partout dans le pipeline
variables:
  NODE_ENV: "test"          # L'environnement Node.js
  npm_config_cache: ".npm"  # Ou stocker le cache npm

# --- CACHE GLOBAL ---
# Permet de sauvegarder des fichiers entre les jobs
# Evite de retelecharger les dependances a chaque fois
cache:
  key: ${CI_COMMIT_REF_SLUG} # Cle unique par branche
  paths:
    - node_modules/          # On met en cache les dependances
    - .npm/                  # Et le cache npm

# --- JOB : LINT ---
lint:
  stage: validate            # Ce job appartient au stage "validate"
  script:
    - npm ci                 # Les commandes a executer
    - npm run lint           # Installe les dependances
                                # Lance la verification du code
  only:
    - merge_requests         # Quand executer ce job ?
    - main                   # Seulement sur les merge requests
                                # Et sur la branche main

# --- JOB : BUILD ---
build:
  stage: build               # Ce job appartient au stage "build"
  script:
    - npm ci                 # Installe les dependances
    - npm run build          # Construit l'application
  artifacts:                  # Fichiers a conserver apres le job
  paths:
    - dist/                  # Le dossier de build
  expire_in: 1 hour          # Garder pendant 1 heure

# --- JOB : TEST ---
test:
```

4.3 Les mots-cles importants

Mot-cle	Description	Exemple
<code>image</code>	Image Docker pour le job	<code>image: node:18</code>
<code>stage</code>	A quel stage appartient le job	<code>stage: test</code>
<code>script</code>	Commandes a executer	<code>script: [npm test]</code>
<code>before_script</code>	Commandes avant le script	<code>before_script: [npm ci]</code>
<code>after_script</code>	Commandes apres le script	<code>after_script: [rm -rf tmp]</code>
<code>variables</code>	Variables d'environnement	<code>variables: {NODE_ENV: test}</code>
<code>only</code>	Quand executer (ancien)	<code>only: [main]</code>
<code>rules</code>	Quand executer (nouveau)	Voir section 12
<code>artifacts</code>	Fichiers a conserver	<code>artifacts: {paths: [dist/]}</code>
<code>cache</code>	Fichiers a mettre en cache	<code>cache: {paths: [node_modules/]}</code>
<code>needs</code>	Dependances entre jobs	<code>needs: [build]</code>
<code>when</code>	Condition d'execution	<code>when: manual</code>
<code>allow_failure</code>	Autoriser l'echec	<code>allow_failure: true</code>
<code>retry</code>	Nombre de tentatives	<code>retry: 2</code>
<code>timeout</code>	Duree max du job	<code>timeout: 10 minutes</code>

5. Le principe Fail Fast

5.1 Qu'est-ce que le Fail Fast ?

Fail Fast signifie "echouer rapidement". L'idee est simple :

si quelque chose ne va pas, on veut le savoir LE PLUS TOT POSSIBLE.

Analogie : Imaginez que vous construisez une maison. Le Fail Fast, c'est comme verifier les fondations AVANT de construire les murs. Si les fondations sont mauvaises, on le sait tout de suite au lieu de s'en rendre compte une fois le toit pose.

5.2 Pourquoi c'est important ?


```
Sans Fail Fast :  
[Lint: 2min] --> [Build: 5min] --> [Test: 10min] --> ECHEC au test  
Total avant de connaitre le probleme : 17 minutes  
  
Avec Fail Fast :  
[Lint: 2min] --> ECHEC au lint  
Total avant de connaitre le probleme : 2 minutes
```

On economise **15 minutes** dans cet exemple. Multipliez par 50 developpeurs qui poussent 5 fois par jour, et vous comprenez l'enjeu.

5.3 Comment appliquer le Fail Fast ?

Regle 1 : Les verifications les plus rapides en premier

```
stages:  
- validate      # Rapide : lint, format (secondes)  
- build         # Moyen : compilation (minutes)  
- test          # Long : tests unitaires, integration (minutes)  
- test-e2e      # Tres long : tests end-to-end (dizaines de minutes)  
- deploy        # Variable : deploiement
```

Regle 2 : Utiliser `needs` pour ne pas attendre inutilement

```
# Sans needs : les jobs d'un stage attendent que TOUS les jobs  
# du stage precedent soient finis  
  
# Avec needs : un job demarre des que ses dependances sont finies  
test-unitaire:  
  stage: test  
  needs: [build]          # Demarre des que "build" est fini  
  script:  
    - npm run test:unit  
  
test-integration:  
  stage: test  
  needs: [build]          # Demarre des que "build" est fini (en parallele du test unitaire)  
  script:  
    - npm run test:integration
```

Regle 3 : Permettre les echecs non critiques

```
lint-style:  
  stage: validate  
  script:  
    - npm run lint:style  
  allow_failure: true      # Ce job peut echouer sans bloquer le pipeline  
  # Utile pour les verifications "nice to have" qui ne sont pas critiques
```

6. La parallelisation

6.1 Pourquoi paralleliser ?

Sans parallelisation :

```
[Test fichier A: 5min] --> [Test fichier B: 5min] --> [Test fichier C: 5min]
Total : 15 minutes
```

Avec parallelisation :

```
[Test fichier A: 5min]
[Test fichier B: 5min]   (en meme temps)
[Test fichier C: 5min]
Total : 5 minutes
```

6.2 Le mot-cle **parallel**:

Le mot-cle **parallel** cree plusieurs copies d'un job qui s'executent en meme temps.

```
# Ce job va creer 5 copies de lui-meme
# Chaque copie recoit un numero (1/5, 2/5, 3/5, 4/5, 5/5)
tests-paralleles:
  stage: test
  parallel: 5           # Cree 5 instances du job
  script:
    - echo "Je suis l'instance $CI_NODE_INDEX sur $CI_NODE_TOTAL"
    - npm run test -- --shard=$CI_NODE_INDEX/$CI_NODE_TOTAL
    # --shard divise les tests en 5 groupes
    # Chaque instance execute un groupe different
```

Variables disponibles :

- **\$CI_NODE_INDEX** : le numero de l'instance (1, 2, 3, 4, 5)
- **\$CI_NODE_TOTAL** : le nombre total d'instances (5)

6.3 Le mot-cle **parallel:matrix**

parallel:matrix permet de lancer un job avec differentes combinaisons de variables.

C'est comme un tableau de multiplication : chaque combinaison cree un job.

```
# Ce job va se lancer pour chaque combinaison de NODE_VERSION et OS
test-multi-versions:
  stage: test
  image: node:${NODE_VERSION}      # Utilise la variable comme image
  parallel:
    matrix:
      - NODE_VERSION: ["16", "18", "20"]  # 3 versions de Node.js
        OS: ["alpine", "bullseye"]        # 2 systemes d'exploitation
  # Resultat : 3 x 2 = 6 jobs differents :
  #   - Node 16 + Alpine
  #   - Node 16 + Bullseye
  #   - Node 18 + Alpine
  #   - Node 18 + Bullseye
  #   - Node 20 + Alpine
  #   - Node 20 + Bullseye
  script:
    - echo "Test avec Node.js $NODE_VERSION sur $OS"
    - npm ci
    - npm test
```

Exemple pratique - Tester sur plusieurs navigateurs :

```
test-navigateurs:
  stage: test
  parallel:
    matrix:
      - BROWSER: ["chrome", "firefox", "safari"]
  script:
    - echo "Test sur le navigateur $BROWSER"
    - npm run test:e2e -- --browser=$BROWSER
```

7. Artifacts et Cache

7.1 La difference fondamentale

C'est une confusion TRES courante chez les debutants. Voici la difference :

	Artifacts	Cache
But	Passer des fichiers entre jobs/stages	Accelerer les jobs
Contenu typique	Code compile, rapports de tests	node_modules, .m2, pip cache
Duree	Expire (configurable)	Persiste entre pipelines
Transfert	Automatique entre stages	Meme branche uniquement
Telechargeable	Oui, depuis l'interface GitLab	Non

Analogie :

- **Artifact** = Le gateau que vous passez au serveur pour qu'il le serve aux clients
- **Cache** = Vos ustensiles de cuisine que vous gardez pour la prochaine fois

7.2 Les Artifacts en detail

```
build:
  stage: build
  script:
    - npm ci          # Installe les dependances
    - npm run build    # Construit l'application (cree le dossier dist/)
  artifacts:
    # --- Quels fichiers conserver ---
    paths:
      - dist/          # Le dossier de build
      - coverage/       # Les rapports de couverture
    # --- Combien de temps les garder ---
    expire_in: 1 week  # Expire apres 1 semaine
    # --- Quand les creer ---
    when: on_success   # Seulement si le job reussit
    # Options : on_success, on_failure, always
    # --- Rapport JUnit (pour voir les tests dans GitLab) ---
    reports:
      junit: test-results.xml  # GitLab affiche les resultats dans l'interface
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
```

Utiliser les artifacts dans un job suivant :

```
build:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/          # dist/ est sauvegarde

test:
  stage: test
  script:
    - ls dist/         # dist/ est automatiquement disponible ici !
    - npm run test     # Les tests peuvent utiliser les fichiers buildes
  # Par default, tous les artifacts des stages precedents sont telecharges
```

7.3 Le Cache en detail

```
# --- Cache global (s'applique a tous les jobs) ---
cache:
  # Cle unique pour le cache
  # Meme cle = meme cache
  key: ${CI_COMMIT_REF_SLUG}      # Cle basee sur la branche
  paths:
    - node_modules/              # Le dossier des dependances
    - .npm/                      # Le cache interne de npm
  policy: pull-push              # Lire et ecrire le cache
  # Options de policy :
  #   pull-push : telecharge et met a jour le cache (default)
  #   pull      : telecharge seulement (ne met pas a jour)
  #   push      : met a jour seulement (ne telecharge pas)

# --- Cache specifique a un job ---
build:
  stage: build
  cache:
    key:
      files:
        - package-lock.json      # La cle change si ce fichier change
        # Astuce : si package-lock.json ne change pas, on reutilise le cache
    paths:
      - node_modules/
    policy: pull                  # Ce job lit seulement le cache
  script:
    - npm ci
    - npm run build
```

7.4 Quand utiliser quoi ?

```
Utiliser ARTIFACTS quand :
- Vous devez passer des fichiers d'un stage a un autre
- Vous voulez conserver des rapports de tests
- Vous voulez que l'equipe puisse telecharger un fichier
- Exemples : dist/, build/, rapport.pdf, test-results.xml

Utiliser CACHE quand :
- Vous voulez acclereler l'installation des dependances
- Les fichiers ne changent pas souvent
- Exemples : node_modules/, .m2/repository/, pip cache
```

8. Extends et Anchors YAML

8.1 Le probleme : la repetition

Sans **extends**, vous devez copier-coller la meme configuration dans chaque job :

```
# MAUVAIS : beaucoup de repetition !
test-unitaire:
  image: node:18-alpine
  before_script:
    - npm ci
  cache:
    paths:
      - node_modules/

test-integration:
  image: node:18-alpine      # Copie !
  before_script:
    - npm ci                # Copie !
  cache:
    paths:
      - node_modules/      # Copie !
```

8.2 Solution 1 : **extends**

Le mot-cle **extends** permet d'heriter de la configuration d'un autre job.

```
# Job "template" (commence par un point = n'est pas execute)
.base-node:
  image: node:18-alpine
  before_script:
    - npm ci
  cache:
    paths:
      - node_modules/

# Les jobs qui heritent du template
test-unitaire:
  extends: .base-node      # Herite de toute la config de .base-node
  stage: test
  script:
    - npm run test:unit    # Ajoute seulement ce qui change

test-integration:
  extends: .base-node      # Herite aussi de .base-node
  stage: test
  script:
    - npm run test:integration

# Resultat : les deux jobs ont image, before_script et cache
# sans avoir a les reecrire !
```

Note : un job dont le nom commence par un **point** (`.base-node`) est un "job cache" (hidden job). Il n'est JAMAIS execute, il sert uniquement de template.

8.3 Solution 2 : Les Anchors YAML (&, *, <<)

Les anchors sont une fonctionnalite native de YAML (pas specifique a GitLab).

```
# Définir un anchor avec & (comme donner un nom a un bloc)
.variables-communes: &variables-communes
  NODE_ENV: "test"
  LOG_LEVEL: "debug"

# Utiliser un anchor avec * (comme copier le bloc)
test-unitaire:
  variables:
    <<: *variables-communes    # Insere toutes les variables
    TEST_TYPE: "unit"         # Ajoute une variable spécifique

test-integration:
  variables:
    <<: *variables-communes    # Meme variables de base
    TEST_TYPE: "integration"   # Mais type différent
```

Autre exemple avec une liste :

```
# Définir un anchor pour une liste de commandes
.install-steps: &install-steps
- apt-get update
- apt-get install -y curl
- npm ci

build:
  script:
    - *install-steps          # Insere toute la liste
    - npm run build           # Ajoute une commande spécifique
```

8.4 Extends vs Anchors : lequel choisir ?

	extends	Anchors YAML
Fusion intelligente	Oui (merge en profondeur)	Non (remplacement simple)
Lisibilité	Meilleure	Moins bonne
Multi-niveaux	Oui (extends peut hériter d'un extends)	Non
Recommandation	A PRIVILEGIER	Pour des cas simples

9. Include : modularisation

9.1 Le problème : un fichier trop gros

Quand votre pipeline grandit, le fichier `.gitlab-ci.yml` peut devenir très long et difficile à maintenir. La solution : le découper en plusieurs fichiers.

9.2 Le mot-cle `include`

```
# .gitlab-ci.yml (fichier principal, a la racine du projet)
# Ce fichier inclut d'autres fichiers de configuration

# Definir les stages
stages:
  - validate
  - build
  - test
  - deploy

# Inclure les fichiers de chaque stage
include:
  # --- Inclusion locale (fichier dans le meme projet) ---
  - local: '.gitlab-ci/validate.yml' # Fichier dans le projet
  - local: '.gitlab-ci/build.yml'    # Fichier dans le projet
  - local: '.gitlab-ci/test.yml'     # Fichier dans le projet
  - local: '.gitlab-ci/deploy.yml'   # Fichier dans le projet

  # --- Inclusion depuis un autre projet GitLab ---
  - project: 'mon-groupe/templates-ci' # Nom du projet distant
    ref: main                          # Branche a utiliser
    file: '/templates/node.yml'        # Chemin dans ce projet

  # --- Inclusion depuis une URL ---
  - remote: 'https://example.com/ci-template.yml'

  # --- Inclusion d'un template GitLab officiel ---
  - template: 'Security/SAST.gitlab-ci.yml'
```

9.3 Organisation recommandee des fichiers

```
mon-projet/
  .gitlab-ci.yml          # Fichier principal (stages + includes)
  .gitlab/
    ci/
      validate.yml        # Jobs de validation (lint, format)
      build.yml           # Jobs de build
      test.yml            # Jobs de test
      deploy.yml          # Jobs de deploiement
  src/                   # Code source
  tests/                 # Tests
```

9.4 Exemple complet


```
# === .gitlab-ci.yml ===
stages:
  - validate
  - build
  - test
  - deploy

include:
  - local: '.gitlab/ci/validate.yml'
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/test.yml'
  - local: '.gitlab/ci/deploy.yml'

# Variables globales disponibles dans tous les fichiers inclus
variables:
  NODE_VERSION: "18"
```

```
# === .gitlab/ci/validate.yml ===
lint:
  stage: validate
  image: node:${NODE_VERSION}-alpine
  script:
    - npm ci
    - npm run lint
```

```
# === .gitlab/ci/build.yml ===
build:
  stage: build
  image: node:${NODE_VERSION}-alpine
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/
```

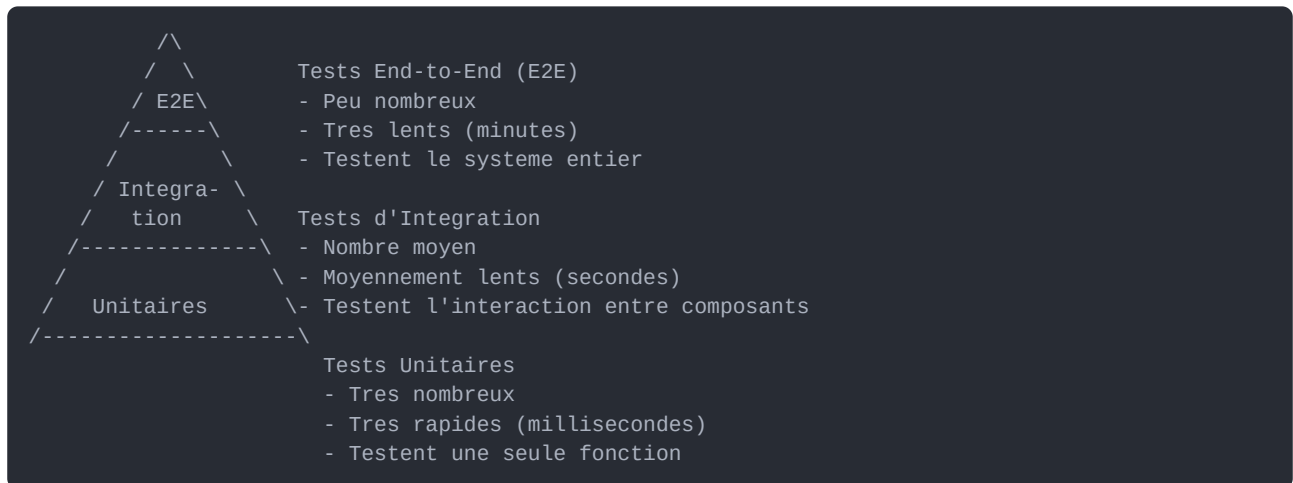
```
# === .gitlab/ci/test.yml ===
test-unitaire:
  stage: test
  image: node:${NODE_VERSION}-alpine
  script:
    - npm ci
    - npm test
```

```
# === .gitlab/ci/deploy.yml ===
deploy-staging:
  stage: deploy
  script:
    - echo "Deploiement staging"
  environment:
    name: staging
```

10. La pyramide de tests

10.1 Qu'est-ce que la pyramide de tests ?

La pyramide de tests est un modele qui guide combien de tests de chaque type vous devriez ecrire.



10.2 Tests Unitaires

Un test unitaire verifie qu'une **seule fonction** fonctionne correctement.

```
// Fonction a tester
function additionner(a, b) {
  return a + b;
}

// Test unitaire
test('additionner 2 + 3 donne 5', () => {
  expect(additionner(2, 3)).toBe(5); // Verifie le resultat
});

test('additionner -1 + 1 donne 0', () => {
  expect(additionner(-1, 1)).toBe(0); // Verifie avec des negatifs
});
```

Caracteristiques :

- Rapides : millisecondes par test
- Isoles : ne dependent de rien d'autre
- Nombreux : des centaines, voire des milliers
- Dans le pipeline : premier type de test a executer

10.3 Tests d'Integration

Un test d'integration verifie que **plusieurs composants fonctionnent ensemble**.

```
// Test d'integration : verifie que l'API et la base de donnees fonctionnent ensemble
test('GET /api/users retourne la liste des utilisateurs', async () => {
  // On appelle l'API (qui elle-meme appelle la base de donnees)
  const response = await request(app).get('/api/users');

  // On verifie que la reponse est correcte
  expect(response.status).toBe(200); // Code HTTP 200
  expect(response.body).toBeInstanceOf(Array); // C'est un tableau
});
```

Caracteristiques :

- Moyennement rapides : secondes par test
- Nécessitent une base de donnees, un serveur, etc.
- Nombre moyen : dizaines a centaines
- Dans le pipeline : apres les tests unitaires

10.4 Tests End-to-End (E2E)

Un test E2E simule un **utilisateur reel** qui clique sur le site.

```
// Test E2E avec Cypress ou Playwright
test('Un utilisateur peut se connecter', async ({ page }) => {
  await page.goto('http://localhost:3000'); // Ouvre le site
  await page.fill('#email', 'test@example.com'); // Tape l'email
  await page.fill('#password', 'motdepasse'); // Tape le mot de passe
  await page.click('button[type="submit"]'); // Clique sur le bouton
  await expect(page).toHaveURL('/dashboard'); // Verifie la redirection
});
```

Caracteristiques :

- Lents : minutes par test
- Fragiles : peuvent echouer pour des raisons non liees au code
- Peu nombreux : dizaines maximum
- Dans le pipeline : en dernier, seulement si tout le reste passe

10.5 Appliquer la pyramide dans GitLab CI

```

stages:
  - validate
  - build
  - test-unit          # Les plus rapides en premier
  - test-integration   # Ensuite les tests d'integration
  - test-e2e           # Les plus lents en dernier
  - deploy

test-unitaire:
  stage: test-unit
  script:
    - npm run test:unit
  # S'exécute en premier : si les tests unitaires échouent,
  # on ne perd pas de temps avec les tests plus longs

test-integration:
  stage: test-integration
  script:
    - npm run test:integration
  # S'exécute seulement si les tests unitaires passent

test-e2e:
  stage: test-e2e
  script:
    - npm run test:e2e
  # S'exécute en dernier : le plus coûteux en temps

```

11. Les patterns de déploiement

11.1 Pourquoi des stratégies de déploiement ?

Déployer une nouvelle version est toujours risqué. Et si la nouvelle version a un bug ?

Les stratégies de déploiement permettent de minimiser ce risque.

11.2 Déploiement classique (Big Bang)

```

Avant : [Ancienne version] ---> Coupure ---> [Nouvelle version]
      |
      | Utilisateurs
      | ne peuvent pas
      | accéder au site

```

- Simple mais risqué
- Temps d'arrêt (downtime) pendant le déploiement
- Si la nouvelle version a un bug : tout le monde est impacté

11.3 Blue/Green Deployment

```

Etape 1 : Les utilisateurs sont sur "Blue" (version actuelle)
[Blue: v1.0] <--- Trafic --- Utilisateurs
[Green: vide]

Etape 2 : On deploye la nouvelle version sur "Green"
[Blue: v1.0] <--- Trafic --- Utilisateurs
[Green: v2.0] (deploiement en cours, invisible pour les utilisateurs)

Etape 3 : On bascule le trafic vers "Green"
[Blue: v1.0]
[Green: v2.0] <--- Trafic --- Utilisateurs

Etape 4 : Si probleme, on rebasculer vers "Blue" instantanement !
[Blue: v1.0] <--- Trafic --- Utilisateurs (rollback instantane)
[Green: v2.0] (version avec probleme)

```

Avantages :

- Zero temps d'arret
- Rollback instantane
- Possibilite de tester Green avant de basculer

Inconvenients :

- Necessite le double de ressources (deux environnements)
- Complexite de gestion des bases de donnees

11.4 Canary Deployment

```

Etape 1 : Tous les utilisateurs sur la version actuelle
[v1.0] <--- 100% du trafic --- Utilisateurs

Etape 2 : 5% du trafic vers la nouvelle version
[v1.0] <--- 95% du trafic --- Utilisateurs
[v2.0] <--- 5% du trafic --- (quelques utilisateurs)

Etape 3 : Si tout va bien, on augmente progressivement
[v1.0] <--- 50% du trafic
[v2.0] <--- 50% du trafic

Etape 4 : Finalement, 100% sur la nouvelle version
[v2.0] <--- 100% du trafic --- Tous les utilisateurs

```

Le nom "Canary" vient des canaris que les mineurs utilisaient dans les mines. Si le canari cessait de chanter, c'était signe de danger. Ici, les 5% d'utilisateurs sont notre "canari" : s'ils ont des problemes, on arrete tout.

Avantages :

- Risque minimal (seulement 5% des utilisateurs touches)
- Detection progressive des problemes
- Possibilite de mesurer les performances

Inconvenients :

- Plus complexe a mettre en place
- Necessite un bon systeme de monitoring

11.5 Rolling Deployment

```

Serveur 1: [v1.0] --> mise a jour --> [v2.0]
Serveur 2: [v1.0]                [v1.0] --> mise a jour --> [v2.0]
Serveur 3: [v1.0]                [v1.0]                [v1.0] --> [v2.0]

Le trafic est toujours distribue entre les serveurs disponibles.
Il y a toujours au moins un serveur qui fonctionne.

```

Avantages :

- Zero temps d'arret
- Pas besoin du double de ressources
- Deploiement progressif

Inconvenients :

- Pendant le deploiement, deux versions coexistent
- Rollback plus lent

11.6 Tableau comparatif

Strategie	Downtime	Rollback	Ressources	Complexite
Big Bang	Oui	Lent	Normal	Faible
Blue/Green	Non	Instantane	Double	Moyenne
Canary	Non	Rapide	Normal + peu	Elevee
Rolling	Non	Moyen	Normal	Moyenne

12. Les rules conditionnelles

12.1 Pourquoi des conditions ?

Vous ne voulez pas tout executer tout le temps :

- Les tests E2E : seulement sur la branche main (trop longs sinon)
- Le deploiement : seulement quand on est pret
- Le lint : sur toutes les branches

12.2 L'ancien systeme : **only** / **except**

```
# only = le job s'execute SEULEMENT si la condition est vraie
deploy:
  script: echo "deploy"
  only:
    - main          # Seulement sur la branche main

# except = le job s'execute SAUF si la condition est vraie
test:
  script: echo "test"
  except:
    - tags          # Pas sur les tags
```

Note : `only/except` est l'ancien systeme. GitLab recommande d'utiliser `rules` a la place.

12.3 Le nouveau systeme : **rules**

rules est plus puissant et plus flexible que **only/except**.

```
deploy-production:
  stage: deploy
  script:
    - echo "Deploiement en production"
  rules:
    # Regle 1 : si c'est la branche main ET un push
    - if: '$CI_COMMIT_BRANCH == "main" && $CI_PIPELINE_SOURCE == "push"'
      when: manual          # Alors : execution manuelle requise

    # Regle 2 : si c'est un tag qui commence par "v"
    - if: '$CI_COMMIT_TAG =~ /^v\d+/'
      when: on_success      # Alors : execution automatique

    # Regle 3 : dans tous les autres cas
    - when: never          # Ne pas executer
```

12.4 Les conditions disponibles

```

rules:
  # --- Condition sur la branche ---
  - if: '$CI_COMMIT_BRANCH == "main"'

  # --- Condition sur le tag ---
  - if: '$CI_COMMIT_TAG'                # Si un tag existe

  # --- Condition sur la source du pipeline ---
  - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
  # Sources possibles : push, merge_request_event, schedule, web, trigger, api

  # --- Condition sur une variable personnalisée ---
  - if: '$DEPLOY_ENV == "production"'

  # --- Condition sur les fichiers modifiés ---
  - changes:
    - src/**/*                # Si un fichier dans src/ a change
    - package.json            # Ou si package.json a change

  # --- Condition "si le fichier existe" ---
  - exists:
    - Dockerfile              # Si le fichier Dockerfile existe

  # --- Combinaisons ---
  - if: '$CI_COMMIT_BRANCH == "main"'
    changes:
    - src/**/*
  when: on_success

```

12.5 Les valeurs de **when**

Valeur	Description
on_success	S'exécute si les jobs précédents ont réussi (défaut)
on_failure	S'exécute si un job précédent a échoué
always	S'exécute toujours, peu importe le résultat
manual	Nécessite un clic dans l'interface GitLab
delayed	S'exécute après un délai (avec start_in)
never	Ne s'exécute jamais

12.6 Exemples pratiques


```
# Exemple 1 : Notifier en cas d'echec
notification-echec:
  stage: deploy
  script:
    - curl -X POST "$SLACK_WEBHOOK" -d '{"text":"Pipeline echoue !"}'
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: on_failure      # Seulement si le pipeline echoue

# Exemple 2 : Deploiement automatique en staging, manuel en production
deploy-staging:
  stage: deploy
  script:
    - echo "Deploiement staging"
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'
      when: on_success      # Automatique sur develop

deploy-production:
  stage: deploy
  script:
    - echo "Deploiement production"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual          # Manuel sur main
      allow_failure: true   # Le pipeline reste vert meme sans deploiement

# Exemple 3 : Executer les tests E2E seulement si le code front a change
test-e2e:
  stage: test-e2e
  script:
    - npm run test:e2e
  rules:
    - changes:
        - frontend/**/*      # Seulement si le code front a change
        - e2e/**/*           # Ou si les tests E2E ont change
```

13. Recapitulatif et checklist

Ce que vous avez appris aujourd'hui

1. **CI/CD** : processus automatique pour integrer, tester et deployer du code
2. **Pipeline** : enchainement de stages executes sequentiellement
3. **Stage** : etape du pipeline contenant des jobs paralleles
4. **Job** : tache individuelle avec un script
5. **Runner** : machine qui execute les jobs
6. **Fail Fast** : detecter les erreurs le plus tot possible
7. **Parallelisation** : executer plusieurs jobs en meme temps
8. **Artifacts** : fichiers passes entre les stages
9. **Cache** : fichiers conserves pour acclerer les jobs
10. **Extends** : heritage de configuration entre jobs
11. **Include** : decoupage du pipeline en plusieurs fichiers

- 12. **Pyramide de tests** : unitaires > integration > E2E
- 13. **Patterns de deployment** : Blue/Green, Canary, Rolling
- 14. **Rules** : conditions pour controler quand un job s'execute

Checklist de validation

Cochez chaque element quand vous l'avez compris et pratique :

- ☐ Je comprends la difference entre CI et CD
- ☐ Je sais ce qu'est un pipeline, un stage, un job et un runner
- ☐ Je sais creer un fichier `.gitlab-ci.yml` de base
- ☐ Je comprends le principe Fail Fast et je sais l'appliquer
- ☐ Je sais utiliser `parallel:` et `parallel:matrix`
- ☐ Je connais la difference entre artifacts et cache
- ☐ Je sais utiliser `extends` pour eviter la repetition
- ☐ Je sais utiliser `include` pour decouper mon pipeline
- ☐ Je comprends la pyramide de tests
- ☐ Je connais les 3 strategies de deployment principales
- ☐ Je sais utiliser `rules` pour controler mes jobs
- ☐ J'ai complete le TP1 (Pipeline Multi-Stage)
- ☐ J'ai complete le TP2 (Blue/Green Deployment)

Ressources pour aller plus loin

- Documentation officielle GitLab CI : <https://docs.gitlab.com/ee/ci/>
- Referentiel complet des mots-cles : <https://docs.gitlab.com/ee/ci/yaml/>
- Exemples de pipelines : <https://docs.gitlab.com/ee/ci/examples/>
- GitLab CI Lint (pour verifier votre syntaxe) : disponible dans votre projet GitLab sous CI/CD > Editor

Bonnes pratiques a retenir

1. **Toujours commenter votre `.gitlab-ci.yml`** : dans 6 mois, vous serez content de comprendre pourquoi vous avez ecrit ca
2. **Fail Fast** : les etapes rapides en premier
3. **DRY (Don't Repeat Yourself)** : utilisez `extends` et `include`
4. **Versionner le pipeline** : le `.gitlab-ci.yml` est dans le depot, donc il est versionne avec le code
5. **Tester localement** : utilisez `gitlab-ci-local` pour tester votre pipeline sans pousser
6. **Securite** : ne mettez JAMAIS de mots de passe ou cles dans le `.gitlab-ci.yml`, utilisez les variables CI/CD de GitLab

****Prochain cours : Jour 2**** - Nous approfondirons Docker, les environnements, et les pipelines avances avec des deploiements reels.